

Optimal Designs for Large Time-Dependent Networks

Justin Lamberti · Kirsten Schorning ·
Christine H. Müller

May 28, 2026

Abstract We consider the problem of optimally allocating measurement devices in complex time-dependent networks so that precise state estimation of the network at a future time-point is possible. For that purpose, we formulate a linear random-effects model in which the network structure and time-dependence can be separated. We then focus on the problem of optimally allocating measurement devices of different precision by reformulating the discrete optimization problem as a continuous one. Using the A -optimality criterion, we formulate an optimality criterion for the optimal state estimation of the network at a future time-point and we provide an analytical solution of the corresponding A -optimal designs. However, calculating the A -optimal design becomes computationally demanding when the network structure is large and complex. In this situation, the network has to be reduced before calculating the A -optimal design. For that purpose, we propose two methods: one using the quotient graph and the other cutting the network into subgraphs that are then treated separately. We show in a simulation study that the designs based on the cutting approach perform well compared to the analytically determined designs in terms of A -efficiencies.

This research was funded in the course of TRR 391 *Spatio-temporal Statistics for the Transition of Energy and Transport* (520388526) by the Deutsche Forschungsgemeinschaft (DFG, German Research Foundation).

J. Lamberti
Department of Statistics, TU Dortmund University, D-44227 Dortmund, Germany,
E-mail: justin.lamberti@tu-dortmund.de, <https://orcid.org/0009-0000-1233-5025>

K. Schorning
Department of Statistics, TU Dortmund University, D-44227 Dortmund, Germany,
E-mail: kirsten.schorning@tu-dortmund.de, <https://orcid.org/0000-0002-9401-5486>

C. H. Müller
Department of Statistics, TU Dortmund University, D-44227 Dortmund, Germany,
E-mail: cmueller@statistik.tu-dortmund.de, <https://orcid.org/0000-0002-4097-3320>

Keywords State space model, time-dependent network, quotient graph, subgraphs, A-optimality

1 Introduction

We provide a new approach to obtain optimal designs for state estimation in large time-dependent networks. Our approach is motivated by state estimation in electrical power distribution grids, but it can also be applied to other state estimation problems in time-dependent networks.

In particular, reliable state estimation and prediction in electrical power distribution grids of medium and low voltage became in recent years a very relevant problem, driven by the increased volatility of loads and infeeds from electric cars, heating pumps, and renewable energy production. Moreover, this increased volatility is strongly time-dependent. However, state estimation and prediction suffer from the fact that only a few grid measurements come from smart meters, which provide very precise data. At other nodes of the grid, less accurate pseudo-measurements derived from weather data or historical load profiles must be included to achieve identifiable states, see, e.g., Schlösser et al (2014). To improve state estimation and prediction, the question arises: which grid nodes require high-precision measurements and which require less precise measurements, i.e., at which nodes the few high-precision sensors should be allocated.

The problem of sensor or generator allocation in electrical power grids was addressed, for example, by Li et al (2011), Xygkis et al (2018), Cao et al (2022), using greedy algorithms due to the high number of positions. A similar problem to the sensor allocation in networks is the treatment allocation in networks. To solve the treatment allocation problem, Koutra et al (2021) provided a modification of the exchange algorithm of Fedorov (2013) and Cook and Nachtsheim (1980), while Bui et al (2025) used a genetic algorithm and Zhang (2025) a rerandomization algorithm. Some theoretical results on optimal sensor precision in simple networks were provided by Müller and Schorning (2023).

However, none of these approaches considers time-dependent networks. Du et al (2020) provided a recursive online optimization of generators in electrical power grids, but time-dependence is not modeled. There are only a few approaches that address the design problem of time-dependent networks. For example, Singhal and Michailidis (2008) used a greedy algorithm for the sampling problem in time-dependent computer-communication networks. Duan et al (2022) determined an optimal sensor scheduling design for cyber-physical systems with linear state-space equations iteratively by reformulating the problem as a Markov decision process. Jiang and Gómez (2025) provided an adaptive sampling strategy for online monitoring of partially observed networks. Other approaches, for example, of Alexanderian (2021), Uciński (2022), Uciński and Patan (2024), Uciński (2025) treated the sensor location problem in spatio-temporal systems modeled by partial differential equations. Alexan-

derian (2021) considered this problem by a Bayesian approach, while Uciński (2022) solved the E-optimum sensor selection by reformulating the problem as a convex semi-infinite programming problem. A-optimal sensor locations were found by Uciński and Patan (2024) via convex relaxation and by Uciński (2025) using a modified exchange algorithm.

Recently, Müller (2026) treated the sensor location problem for state estimation of time-dependent electrical power distribution grids by determining the nodes, where high precision of the measurements is necessary, and the nodes, where less precise measurements are sufficient. She used simplified criteria appearing in a nonlinear Kalman filter and considered simple small networks. Here, we present an alternative approach that uses different simplifications for a similar problem and can handle large, complex networks.

In Section 2, we extend the static network model for state estimation in Müller and Schorning (2023) by incorporating a separable time-dependence. This allows for a state estimation via a classical least squares estimator and leads to a design problem of minimizing the trace of the estimator's covariance matrix independently of the time-dependence. Section 4 presents two novel methods for solving this design problem in large networks. One of these methods uses only the quotient graph. The second method splits the entire network in subgraphs and treats the design problem separately for the subgraphs. Since both methods are based on special graph theoretical results, we present some foundations of graph theory in Section 3. In Section 5, we evaluate the performance of the different methods using a simulation study. In particular, we consider the case of large networks, for which we can calculate the A-optimal design analytically using the approach of Müller and Schorning (2023). We compare the designs obtained by the new methods to the analytically determined optimal design using the A-efficiencies. It turns out that the A-efficiencies of the designs based on the reduction algorithms are high. A summary and a discussion of the results are given in Section 6.

2 The model and the design problem

We consider a network with $I \in \mathbb{N}$ nodes, which could describe I house holds in an electrical power distribution grid. At each node $i \in \{1, \dots, I\}$, N measurements $y_{i1}, \dots, y_{iN}$ are available that are measured at time points $t_1 < \dots < t_N \in [0, T]$. The measurement value y_{in} is influenced by the random state $S_i(t_n)$ of the node i at the time point t_n and the random states $S_k(t_n)$ of the adjacent nodes $k \neq i$.

Additionally, there is a device-specific measurement error ε_{in} at the i -th node. Consequently, we consider the model

$$Y_{in} = S_i(t_n) + \sum_{\substack{k=1 \\ k \neq i}}^I \alpha_{ik} S_k(t_n) + E_{in} \text{ for } i \in \{1, \dots, I\}; n \in \{1, \dots, N\}, \quad (1)$$

where E_{in} are independent random variables with $\mathbb{E}[E_{in}] = 0$ and $\mathbb{V}ar(E_{in}) = \sigma_i^2 \sigma^2 < \infty$ for $i = 1, \dots, N$ and α_{ik} describes the influence of the k -th node on the i -th node. Note that we assume that the nodes' influence does not change over time. The variance term σ^2 models a common variance, and the variances $\sigma_1^2, \dots, \sigma_I^2$ concern different variances of the measurement devices at the I different nodes.

Let the matrix $\mathbf{I}_{I \times I}$ be the $I \times I$ identity matrix and the matrix $A = (\alpha_{ik})_{i,k=1,\dots,I} \in \mathbb{R}^{I \times I}$ be the matrix that describes the influence of the adjacent nodes. Then we can define the $I \times I$ -dimensional influence matrix by

$$\mathbb{X} = \mathbf{I}_{I \times I} + A$$

and rewrite model (1) by

$$\mathbf{Y}_n = \mathbb{X}\mathbf{S}(t_n) + \mathbf{E}_n \text{ for } n \in \{1, \dots, N\} \quad (2)$$

where the I -dimensional random vector $\mathbf{Y}_n = (Y_{1n}, \dots, Y_{In})^\top$ consists of the I random variables at the different nodes of the network. The random states are described by

$$\mathbf{S}(t_n) = \mathbf{s}(t_n) + \mathbf{Z}_n, \quad (3)$$

where $\mathbf{s}(t_n) = (\mathbf{s}_1(t_n), \dots, \mathbf{s}_I(t_n))^\top$ are the expected states and $\mathbf{Z}_n$ describes the randomness for the states. It is assumed that $\mathbf{Z}_n$ has mean $\mathbf{0}_I$ and covariance $\sigma^2 \mathbf{D}_Z$, and is independent of $\mathbf{E}_n$.

Note that model (1) extends Model B of Müller and Schorning (2023) by allowing the states to depend on the time t_n . As in Müller and Schorning (2023), the aim is to estimate the expected values of the network's states. However, now the states are time-dependent, so the aim is to estimate $s(t_n)$ for all time points $t_1, \dots, t_N$.

As in Müller and Schorning (2023), the design problem is to determine the device-specific variances $\sigma_1^2, \dots, \sigma_I^2$ of the measurements at the I nodes so that the precision of the estimated states is as good as possible. Since the inverse of the variance of a measurement is its precision δ_i , we set $\delta_i := \frac{1}{\sigma_i^2}$, $\delta = (\delta_1, \dots, \delta_I)^\top$, and consider the design region $\Delta_c := \{\delta \in (0, 1)^I \mid \sum_{i=1}^I \delta_i \leq c\}$, where c is some precision bound or reference precision. Since we use the common variance σ^2 , we can set, without loss of generality, $c = 1$ and consider

$$\Delta := \Delta_1 := \{\delta \in (0, 1)^I \mid \sum_{i=1}^I \delta_i \leq 1\} \quad (4)$$

as design region.

This problem, for example, arises in electrical power distribution grids when estimating the voltage magnitude at the nodes. At a few nodes, voltage magnitude is measured with smart meters with very high precision, whereas at other nodes they are measured using pseudo-measurements obtained by weather data or historical load profiles, which have very low precision. Since the

number of the smart meters is limited, there is a precision bound $c_* = c/(I\sigma^2)$ so that the average of the precision of the measurements are bounded by c_* .

The above design region is the same as in Müller (2026), but the states and the model are different. In Müller (2026), the interest does not only lie in the voltage magnitudes at the nodes of the grid but also in the real and imaginary parts of the voltages, which are also important for grid stability. These voltage states can be obtained via a nonlinear relationship between the real and imaginary voltage values and the real and imaginary power values, which are also measured by smart meters. Because of this nonlinear relationship, an extended Kalman filter was used for estimating the voltage states in Müller (2026). This also allows for the inclusion of time dependence of the states via an autoregressive process. Since this approach leads to very complex design measures, we present a time-dependent approach that is easier to handle in the next section.

2.1 The time-dependent network model

Using the equations (2) and (3), we obtain the following random effects model:

$$\mathbf{Y}_n = \mathbb{X}\mathbf{S}(t_n) + \mathbf{E}_n = \mathbb{X}\mathbf{s}(t_n) + \mathbb{X}\mathbf{Z}_n + \mathbf{E}_n = \mathbb{X}\mathbf{s}(t_n) + \boldsymbol{\varepsilon}_n,$$

where $\sigma^2 \mathbf{D}_{\mathbf{E}}$ is the covariance matrix of $\mathbf{E}_n$ and $\mathbf{D}_{\mathbf{E}}$ is a diagonal matrix with diagonal elements $\sigma_1^2, \dots, \sigma_I^2$. Since the random vector $\mathbf{E}_n$ is independent of the error vector $\mathbf{Z}_n$ of the random states, the covariance matrix of $\mathbf{Y}_n$ is $\boldsymbol{\Sigma} = \sigma^2 (\mathbb{X}\mathbf{D}_{\mathbf{Z}}\mathbb{X}^\top + \mathbf{D}_{\mathbf{E}})$, so that the model can be rewritten by

$$\tilde{\mathbf{Y}}_n := \boldsymbol{\Sigma}^{-1/2}\mathbf{Y}_n = \boldsymbol{\Sigma}^{-1/2}\mathbb{X}\mathbf{s}(t_n) + \boldsymbol{\Sigma}^{-1/2}\boldsymbol{\varepsilon}_n = \mathbf{X}\mathbf{s}(t_n) + \tilde{\boldsymbol{\varepsilon}}_n$$

with $\mathbf{X} = \boldsymbol{\Sigma}^{-1/2}\mathbb{X}$. The covariance of $\tilde{\mathbf{Y}}_n$ and $\tilde{\boldsymbol{\varepsilon}}_n$ is given by $\mathbf{I}_{I \times I}$. It is assumed that the gap between consecutive observations is sufficiently large, so it is reasonable to treat the observations as mutually independent. Let $\tilde{\mathbf{Y}}$ be defined as $\tilde{\mathbf{Y}} := (\tilde{\mathbf{Y}}_1^\top, \dots, \tilde{\mathbf{Y}}_N^\top)^\top$, so that we obtain the following model

$$\tilde{\mathbf{Y}} = (\mathbf{I}_{N \times N} \otimes \mathbf{X}) \begin{pmatrix} \mathbf{s}(t_1) \\ \vdots \\ \mathbf{s}(t_N) \end{pmatrix} + \begin{pmatrix} \tilde{\boldsymbol{\varepsilon}}_1 \\ \vdots \\ \tilde{\boldsymbol{\varepsilon}}_N \end{pmatrix} = (\mathbf{I}_{N \times N} \otimes \mathbf{X}) \begin{pmatrix} \mathbf{s}(t_1) \\ \vdots \\ \mathbf{s}(t_N) \end{pmatrix} + \tilde{\boldsymbol{\varepsilon}}, \quad (5)$$

where $(\mathbf{s}^\top(t_1), \dots, \mathbf{s}^\top(t_N))^\top \in \mathbb{R}^{IN}$. The error vector $\tilde{\boldsymbol{\varepsilon}}$ of this model satisfies

$$\mathbb{E}[\tilde{\boldsymbol{\varepsilon}}] = \mathbf{0}_{IN}, \quad \text{Cov}(\tilde{\boldsymbol{\varepsilon}}) = \mathbf{I}_{N \times N} \otimes \mathbf{I}_{I \times I} = \mathbf{I}_{IN \times IN}.$$

Hence, the model is a linear model with homoscedastic errors. Furthermore, we assume that the states $\mathbf{s}(t)$ follow a functional dependence over time. This assumption is made to model seasonality or daily variation. Therefore, the state $s_i(t)$ at the i -th node can be described via the product of a vector of base functions $\mathbf{F}_i(t)$ and an unknown parameter vector $\boldsymbol{\theta}_i \in \mathbb{R}^{p_i}$ for $i \in \{1, \dots, I\}$, i.e. $s_i(t) = \mathbf{F}_i^\top(t)\boldsymbol{\theta}_i$. Typical vectors of base functions can be

given by $(1, t, \cos(t))^\top$, the classical polynomials $(1, t^1, \dots, t^K)^\top$, or the base functions of Fourier polynomials. Note that the later ones can be used to approximate any periodical function, with $\cos$ and $\sin$ as basis functions. In particular, using these basis functions enables a quite flexible modeling of the functional behavior of the states at the different nodes with the same set of basis functions. For a detailed explanation about Fourier polynomials and series, see the Supplementary Material . Summarizing, the state vector $\mathbf{s}(t)$ of the complete network at timepoint t is modelled by

$$\begin{aligned} \mathbf{s}(t) &= \begin{pmatrix} s_1(t) \\ \vdots \\ s_I(t) \end{pmatrix} = \begin{pmatrix} \mathbf{F}_1^\top(t) \boldsymbol{\theta}_1 \\ \vdots \\ \mathbf{F}_I^\top(t) \boldsymbol{\theta}_I \end{pmatrix} \\ &= \begin{pmatrix} \mathbf{F}_1^\top(t) & 0_{p_2}^\top & \cdots & 0_{p_I}^\top \\ 0_{p_1}^\top & \mathbf{F}_2^\top(t) & \ddots & \vdots \\ \vdots & \ddots & \ddots & 0_{p_I}^\top \\ 0_{p_1}^\top & \cdots & 0_{p_{I-1}}^\top & \mathbf{F}_I^\top(t) \end{pmatrix} \begin{pmatrix} \boldsymbol{\theta}_1 \\ \vdots \\ \boldsymbol{\theta}_I \end{pmatrix} = \mathbf{F}^\top(t) \boldsymbol{\theta}, \end{aligned}$$

with $\boldsymbol{\theta} \in \mathbb{R}^{(\sum_i p_i)}$ and $\mathbf{F}^\top(t) \in \mathbb{R}^{I \times (\sum_i p_i)}$.

Using Fourier polynomials, we now assume that the states in all nodes follow the same functional relationship described by the vector $\mathbf{f}(t)$ of base functions. Therefore, we obtain $p_i = p$ for all $i \in \{1, \dots, I\}$ and $\mathbf{F}_1(t) = \dots = \mathbf{F}_I(t) =: \mathbf{f}(t)$. Consequently, the matrix of the base functions reduces to $\mathbf{F}^\top(t) = \mathbf{I}_{I \times I} \otimes \mathbf{f}^\top(t)$. Inserting this functional relationship into model (5), it reduces to

$$\begin{aligned} \tilde{\mathbf{Y}} &= (\mathbf{I}_{N \times N} \otimes \mathbf{X}) \begin{pmatrix} (\mathbf{I}_{I \times I} \otimes \mathbf{f}^\top(t_1)) \boldsymbol{\theta} \\ \vdots \\ (\mathbf{I}_{I \times I} \otimes \mathbf{f}^\top(t_N)) \boldsymbol{\theta} \end{pmatrix} + \tilde{\boldsymbol{\varepsilon}} = \begin{pmatrix} \mathbf{X} \otimes \mathbf{f}^\top(t_1) \\ \vdots \\ \mathbf{X} \otimes \mathbf{f}^\top(t_N) \end{pmatrix} \boldsymbol{\theta} + \tilde{\boldsymbol{\varepsilon}} \quad (6) \\ &=: \mathbf{G}^\top(t_1, \dots, t_N) \boldsymbol{\theta} + \tilde{\boldsymbol{\varepsilon}}. \end{aligned}$$

Note that the matrix $\mathbf{G}^\top(t_1, \dots, t_N) \in \mathbb{R}^{IN \times pI}$ is the design matrix, which depends on the measuring time points $t_1, \dots, t_N$. According to the Gauss-Markov Theorem, the best linear unbiased estimator for $\tilde{\mathbf{L}} \boldsymbol{\theta}$, with an appropriate linear aspect $\tilde{\mathbf{L}} \in \mathbb{R}^{s \times pI}$, is given by

$$\tilde{\mathbf{L}} \hat{\boldsymbol{\theta}} = \tilde{\mathbf{L}} (\mathbf{G}(t_1, \dots, t_N) \mathbf{G}^\top(t_1, \dots, t_N))^- \mathbf{G}(t_1, \dots, t_N) \tilde{\mathbf{Y}}.$$

where A^- denotes the generalized inverse of the square matrix A .

In the situation under consideration, we are particularly interested in predicting the state of the network at a specific future time-point $t_{N+1} > T$. Generally speaking a linear combination of the state at time point t_{N+1} is given by $\mathbf{L} \mathbf{s}(t_{N+1})$, where $\mathbf{L} \in \mathbb{R}^{s \times I}$. Using the assumption on the structure of $\mathbf{s}(t)$, this results in

$$\mathbf{L}s(t) = \mathbf{L}(\mathbf{I}_{I \times I} \otimes \mathbf{f}^\top(t))\boldsymbol{\theta} = \tilde{\mathbf{L}}\boldsymbol{\theta}, \quad (7)$$

where $\tilde{\mathbf{L}}$ is the corresponding linear aspect in terms of the parameter vector $\boldsymbol{\theta}$. Thus, a prediction of $\mathbf{L}s(t_{N+1})$ is obtained by plugging the least square estimator $\hat{\boldsymbol{\theta}}$ in (7) resulting in

$$\mathbf{L}\hat{\mathbf{s}}(t) = \mathbf{L} \left[\sum_{j=1}^N \left(\left[\mathbf{f}^\top(t) \left(\sum_{i=1}^N \mathbf{f}(t_i) \mathbf{f}^\top(t_i) \right)^{-1} \mathbf{f}(t_j) \right] \left((\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{X}^\top \tilde{\mathbf{Y}}_j \right) \right) \right]. \quad (8)$$

Note that the covariance matrix of the estimator $\mathbf{L}\hat{\mathbf{s}}(t)$ is given by

$$\text{Cov}(\mathbf{L}\hat{\mathbf{s}}(t)) = (\mathbf{f}^\top(t) \left(\sum_{n=1}^N \mathbf{f}(t_n) \mathbf{f}^\top(t_n) \right)^{-1} \mathbf{f}(t)) \cdot (\mathbf{L}(\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{L}^\top). \quad (9)$$

2.2 DoE problem on time-dependent networks

The model in (6) is now used as reference model to derive the problem on time-dependent networks. Our goal is to accurately estimate the expected states at the different nodes of the network. Therefore, the A -optimality criterion that aims for minimizing the trace of the covariance matrix of the corresponding estimator is an appropriate choice. In the model under consideration, the trace of the covariance matrix (9) is given by

$$\text{tr}(\text{Cov}(\mathbf{L}\hat{\mathbf{s}}(t))) = (\mathbf{f}^\top(t) \left(\sum_{n=1}^N \mathbf{f}(t_n) \mathbf{f}^\top(t_n) \right)^{-1} \mathbf{f}(t)) \text{tr}(\mathbf{L}(\mathbf{X}^\top \mathbf{X})^{-1} \mathbf{L}^\top), \quad (10)$$

it reduces to the product of a one-dimensional time-dependent term and the trace of a spatially dependent term. Consequently, optimization with respect to the time-points and with respect to the position of the sensors can be considered separately.

We first consider the optimization with respect to the time-points. In particular, one-dimensional time-dependent term can be minimized using the constraints $0 \leq t_1 < \dots < t_N \leq T$ while ensuring that the matrix $\sum_{n=1}^N \mathbf{f}(t_n) \mathbf{f}^\top(t_n)$ is non-singular which means in particular $N \geq p$.

The spatially dependent part of the covariance trace in (10) depends on the matrix $\mathbf{X} = \boldsymbol{\Sigma}^{-1/2} \mathbb{X}$ where $\boldsymbol{\Sigma} = \sigma^2 (\mathbb{X} \mathbf{D}_Z \mathbb{X}^\top + \mathbf{D}_E)$ holds. Therefore, the covariance directly depends on the matrix that reflect the precision of the applied measuring methods used at the different nodes of the network. As stated in (4), the accuracies of the applied measurement device in node i is determined by $\delta_i := \frac{1}{\sigma_i^2}$ under the constraint $\sum_{i=1}^N \delta_i \leq 1$.

Setting $\mathbf{D}_\delta := \text{diag}(\delta_1, \dots, \delta_I) = \mathbf{D}_\mathbf{E}^{-1}$, the design problem has the following form

$$\begin{aligned} \delta^* &= \arg \min_{\delta \in \Delta} \left\{ \text{tr} \left(\mathbf{L} [\mathbf{X}^\top \mathbf{X}]^{-1} \mathbf{L}^\top \right) \right\} = \arg \min_{\delta \in \Delta} \left\{ \text{tr} \left(\mathbf{L} [\mathbb{X}^\top \boldsymbol{\Sigma}^{-1} \mathbb{X}]^{-1} \mathbf{L}^\top \right) \right\} \\ &= \arg \min_{\delta \in \Delta} \left\{ \text{tr} \left(\mathbf{L} [\mathbb{X}^\top (\mathbb{X} \mathbf{D}_\mathbf{Z} \mathbb{X}^\top + \mathbf{D}_\delta^{-1})^{-1} \mathbb{X}]^{-1} \mathbf{L}^\top \right) \right\}. \end{aligned} \quad (11)$$

For non-random states, i.e. $\mathbf{D}_\mathbf{Z} = \mathbf{0}_{I \times I}$, it follows $\boldsymbol{\Sigma}^{-1} = \frac{1}{\sigma^2} \mathbf{D}_\delta$ and thus

$$\delta^* = \arg \min_{\delta \in \Delta} \left\{ \text{tr} \left(\mathbf{L} [\mathbb{X}^\top \mathbf{D}_\delta \mathbb{X}]^{-1} \mathbf{L}^\top \right) \right\}. \quad (12)$$

If $\mathbb{X}$ is non-singular, the same design problem of (12) holds also for random states with an arbitrary given covariance matrix $\sigma^2 \mathbf{D}_\mathbf{Z}$ independent of δ , since then the problem (11) reduces to

$$\begin{aligned} \delta^* &= \arg \min_{\delta \in \Delta} \left\{ \text{tr} \left(\mathbf{L} \mathbb{X}^{-1} (\mathbb{X} \mathbf{D}_\mathbf{Z} \mathbb{X}^\top + \mathbf{D}_\delta^{-1}) (\mathbb{X}^\top)^{-1} \mathbf{L}^\top \right) \right\} \\ &= \arg \min_{\delta \in \Delta} \left\{ \text{tr} \left(\mathbf{L} [\mathbb{X}^\top \mathbf{D}_\delta \mathbb{X}]^{-1} \mathbf{L}^\top \right) \right\}. \end{aligned} \quad (13)$$

Note that Müller and Schorning (2023) observed a similar reduction for the case of time-independent networks.

Then *Proposition 1* of Müller and Schorning (2023) provides an analytical solution to the design problem described in (11) for non-random and random states.

Theorem 1 *If $\mathbb{X}$ is invertible, then $\delta^* = (\delta_1^*, \dots, \delta_I^*)^\top$ is an $A_\mathbf{L}$ -optimal design and therefore a solution to the DoE problem described in (13) if and only if*

$$\delta_i^* = \frac{\sqrt{v_i}}{\sum_{j=1}^I \sqrt{v_j}}, \quad \text{with } v_i = \mathbf{u}_i^\top (\mathbb{X}^{-1})^\top \mathbf{L}^\top \mathbf{L} \mathbb{X}^{-1} \mathbf{u}_i \text{ for } i \in \{1, \dots, I\},$$

where $\mathbf{u}_i \in \mathbb{R}^I$ denotes the i -th unit vector.

Theorem 1 results in the following corollary that takes into account the full state vector and the symmetry of $\mathbb{X}$ for undirected graphs.

Corollary 1 *For $\mathbf{L} = \mathbf{I}_{I \times I}$ the optimal weights δ_i^* are proportional to square root of the i -th diagonal entry of $(\mathbb{X} \mathbb{X}^\top)^{-1}$. If $\mathbb{X}$ is symmetric, which is the case for undirected graphs, then δ_i^* is proportional to the square root of the i -th diagonal entry of $\mathbb{X}^{-2}$.*

3 Brief introduction to graphs

The considered time-dependent networks are motivated by real-world power distribution grids that are complex and have a lot of nodes. In this case, analytically calculating optimal designs and thus determining the optimal allocation of the measurement devices within the network becomes computationally demanding. Thus, the complexity of the network, particularly its size, has to be reduced before determining the A -optimal designs.

Mathematically speaking the considered networks are graphs. Our target is to use concepts of graph theory to provide a method with which the complexity of networks can be reduced. In the following section, we introduce some basic concepts of graph theory that are necessary for understanding the developed network-reduction methods. The concepts presented in this section are motivated by the work of Koutra (2017, pp.77-83), which builds on the definitions put forth by Harary (1969). Some definitions are based on the book of Rahman (2017, pp.11-29). All figures in the examples presented in this section and of Section 4 are made using the software *R*.

3.1 Structure of a network

We consider a graph $\mathcal{G} = (\mathcal{V}, \mathcal{E})$, where $\mathcal{V}$ is a finite set of vertices and $\mathcal{E} \subseteq \mathcal{V} \times \mathcal{V}$ is a finite set of edges connecting two vertices. Throughout, we assume the graph $\mathcal{G}$ has $I \in \mathbb{N}$ vertices, which are labeled by the natural numbers from one to I , such that $\mathcal{V} = \{1, \dots, I\}$. A graph is said to be *undirected* if the edge set $\mathcal{E}$ is symmetric, that is, for every edge $(u, v) \in \mathcal{E}$, the pair (v, u) also belongs to $\mathcal{E}$. An edge of the form (u, u) , connecting a vertex to itself, is called a *loop*. A graph is called *weighted* if each edge has an associated non-zero real-valued weight. In this case, the graph is defined as the triple $\mathcal{G} = (\mathcal{V}, \mathcal{E}, l)$, where $l : \mathcal{E} \rightarrow \mathbb{R} \setminus \{0\}$ assigns a weight to each edge. A graph $\mathcal{G}$ has *multiple edges* if some vertex pair is connected by more than one edge. A graph without loops or multiple edges is referred to as *simple*. A graph is called *connected* if, for every pair of vertices, there exists a path (i.e., a sequence of edges) linking them. Unless stated otherwise, all graphs in this work are assumed to be undirected, simple, and connected.

The $(I \times I)$ -matrix $\mathcal{A} = (a_{i,j})$, whose entries are given by

$$a_{i,j} := \mathbb{1}_{\mathbb{R} \setminus \{0\}}(\alpha_{i,j}) = \begin{cases} 1 & \text{if } (i, j) \in \mathcal{E}, \\ 0 & \text{otherwise.} \end{cases}$$

is called *adjacency matrix* of the graph $\mathcal{G} = (\mathcal{V}, \mathcal{E})$ and provides a matrix representation of its edge set $\mathcal{E}$. Note that this connects the influence values of the adjacent nodes in the model defined in (1) to the entries of the adjacency matrix $\mathcal{A}$. If the graph $\mathcal{G}$ is undirected, the adjacency matrix $\mathcal{A}$ is symmetric, as $(i, j) \in \mathcal{E}$ implies $(j, i) \in \mathcal{E}$. The *degree of a vertex* of a vertex $v \in \mathcal{V}$ in an

undirected graph is given by

$$\deg(v) := \sum_{i=1}^I a_{v,i} = \sum_{i=1}^I a_{i,v}.$$

As the graph is assumed to be connected, the degree of each vertex is at least one. The *degree matrix* is defined as $\mathcal{D} := \text{diag}(\deg(1), \dots, \deg(I))$.

Following Umeyama (1988), the adjacency matrix can also be formulated for weighted graphs $\mathcal{G} = (\mathcal{V}, \mathcal{E}, l)$. More precisely, the *weighted adjacency matrix* $\mathcal{A}_w = (\alpha_{i,j})_{i,j=1,\dots,I} \in \mathbb{R}^{I \times I}$ is defined by

$$\alpha_{i,j} = \begin{cases} l(e) & \text{if } (i,j) = e \in \mathcal{E}, \\ 0 & \text{otherwise.} \end{cases}$$

The matrix $\mathcal{A}_w$ is symmetric if and only if the weight function l is symmetric, $l(i,j) = l(j,i)$ for all $(i,j) \in \mathcal{E}$. From now on, it is assumed that l is symmetric, thus $\mathcal{A}_w$ is symmetric. This definition links up graph theory and our model.

3.2 Permutations of graphs and symmetry

Let η be a bijective mapping from a set of vertices $\mathcal{V}_1$ onto another set of vertices $\mathcal{V}_2$. Then η is called a *vertex permutation*, resulting in $\eta : \mathcal{V}_1 \rightarrow \mathcal{V}_2, v \rightarrow \eta(v)$. Two graphs $\mathcal{G}_1 = (\mathcal{V}_1, \mathcal{E}_1)$ and $\mathcal{G}_2 = (\mathcal{V}_2, \mathcal{E}_2)$ are called *isomorphic*, in notation $\mathcal{G}_1 \cong \mathcal{G}_2$, if and only if a vertex permutation $\eta : \mathcal{V}_1 \rightarrow \mathcal{V}_2$ exists such that $(u,v) \in \mathcal{E}_1 \Leftrightarrow (\eta(u), \eta(v)) \in \mathcal{E}_2$ holds. If $\mathcal{G}_1 \cong \mathcal{G}_2$, the corresponding vertex permutation η is called an *isomorphism*. Suppose $\mathcal{G}_1 \cong \mathcal{G}_2$ under the isomorphism η and the corresponding adjacency matrices are given by $\mathcal{A}_1$ and $\mathcal{A}_2$. Then, there exists a permutation matrix $\mathbf{P}_\eta \in \{0,1\}^{I \times I}$ such that $\mathcal{A}_1 = \mathbf{P}_\eta \mathcal{A}_2 \mathbf{P}_\eta^\top$. The permutation matrix $\mathbf{P}_\eta$ is constructed using the I -dimensional unit vectors $\mathbf{u}_i$, where each $\mathbf{u}_i$ has a one at the i -th entry and zero elsewhere.

Let $\eta : \mathcal{V}_1 \rightarrow \mathcal{V}_2$ be an isomorphism. η is called an *automorphism* if η projects $\mathcal{G}_1$ on itself. Therefore, an automorphism is an isomorphism that exchanges pairs of vertices while preserving the graph's adjacency structure. Let $\mathcal{A}_\eta$ denote the adjacency matrix of a graph $\mathcal{G}$ under the automorphism η , then $\mathcal{A} = \mathcal{A}_\eta$ follows. That is, there exists a permutation matrix $\mathbf{P}_\eta$ that satisfies $\mathbf{P}_\eta \mathcal{A} \mathbf{P}_\eta^\top = \mathcal{A}$. If the graph is a weighted graph, this property can be extended to define an automorphism on weighted graphs: If $\mathcal{A}_w$ is a weighted adjacency of the graph $\mathcal{G} = (\mathcal{V}, \mathcal{E}, l)$ and $\mathcal{A}_{w,\eta}$ the weighted adjacency matrix under a vertex permutation η , then η is called an automorphism if and only if $\mathcal{A}_w = \mathcal{A}_{w,\eta}$ holds.

Applying the automorphism to an edge $e = (i,j)$ means applying the automorphism to both vertices $i, j \in \mathcal{V}$ that the edge connects. Using the adjacency preservation property of automorphisms, the *automorphism group* of a graph $\mathcal{G} = (\mathcal{V}, \mathcal{E}, l)$ is the set of all automorphisms of the corresponding graph and is defined by $\text{Aut}(\mathcal{G}) := \{\eta \mid \eta(\mathcal{E}) = \mathcal{E} \text{ and } l(\eta(e)) = l(e) \text{ for all } e \in \mathcal{E}\}$. The vertex permutation, $\eta : \mathcal{V} \rightarrow \mathcal{V}, v \rightarrow v$, that maps all vertices onto itself is

called the identity. The identity is an automorphism since $\mathcal{G}$ stays unchanged under η , thus fulfilling the edge-preserving property of an automorphism. A graph $\mathcal{G}$ is called *symmetric* if there exists at least one automorphism η on the graph $\mathcal{G}$ that is not the identity. Otherwise, the graph $\mathcal{G}$ is called *asymmetric*.

A *vertex orbit* is an equivalence class for a vertex $v \in \mathcal{V}$ under the automorphisms in $\text{Aut}(\mathcal{G})$. Formally, this can be expressed as

$$\text{orb}_{\mathcal{G}}(v) := \{\eta(v) \mid \eta \in \text{Aut}(\mathcal{G})\}.$$

From now on, it is assumed that all graphs are symmetric, meaning there exists at least one automorphism that is not the identity, and thus at least one vertex orbit contains more than one vertex.

3.3 Subgraphs

Consider a graph $\mathcal{G} = (\mathcal{V}, \mathcal{E}, l)$. $\mathcal{G}' = (\mathcal{V}', \mathcal{E}', l|_{\mathcal{E}'})$ is called a *subgraph* of $\mathcal{G}$ and $\mathcal{G}$ a *supergraph* of $\mathcal{G}'$ if $\mathcal{V}' \subseteq \mathcal{V}$, $\mathcal{E}' \subseteq \mathcal{E}$, and $l|_{\mathcal{E}'}$ is the weight function l restricted to the set $\mathcal{E}'$. The *skeleton* of a symmetric graph $\mathcal{G}$ is a subgraph consisting only of vertices that have a trivial vertex orbit. If the skeleton is denoted by $\mathcal{G}_S = (\mathcal{V}_S, \mathcal{E}_S, l|_{\mathcal{E}_S})$, then the vertices are given by $\mathcal{V} \supset \mathcal{V}_S := \{v \mid v \in \mathcal{V} \wedge \dim(\text{orb}_{\mathcal{G}}(v)) = 1\}$ and the edges are given as $\mathcal{E} \supset \mathcal{E}_S := \{(u, v) \mid (u, v) \in \mathcal{E} \wedge u, v \in \mathcal{V}_S\}$. In the skeleton, we include only vertices that no automorphism of the automorphism group can exchange them. Thus, the skeleton represents the non-interchangeable core of the graph.

Suppose that only two vertices $v_1, v_2 \in \mathcal{V}$ lie in the same vertex orbit, then $\text{orb}_{\mathcal{G}}(v_1) = \{v_1, v_2\} = \text{orb}_{\mathcal{G}}(v_2)$ holds. When calculating the I vertex orbits, the vertex orbit $\{v_1, v_2\}$ is obtained twice. When these duplicate sets are removed, the obtained subset is called the *unique set of vertex orbits*.

Consider a graph $\mathcal{G} = (\mathcal{V}, \mathcal{E}, l)$. The *quotient graph* is a subgraph of $\mathcal{G}$ that consists of one chosen representative from each set of the unique vertex orbits. The vertices are chosen in such a way that they ensure the quotient graph being connected. Consequently, the quotient graph consists of the skeleton and one representative of each unique non-trivial vertex orbit. A quotient graph is obtained by the following procedure. First, the I vertex orbits are calculated, which results in $q \leq I$ unique sets of vertex orbits, which are denoted by $\{\text{orb}_1, \dots, \text{orb}_q\}$. Suppose that the first k vertex orbits contain only one vertex, thus $\dim(\text{orb}_1) = \dots = \dim(\text{orb}_k) = 1$. These k vertices build the skeleton and are denoted by the set $\mathcal{V}_S$. Let $o_{k+1}, \dots, o_q$ be representatives of the corresponding sets $\{\text{orb}_{k+1}, \dots, \text{orb}_q\}$, ensuring that, if there are adjacent vertices in different vertex orbits, both are chosen as representatives of their respective orbits. Consequently, the vertices of the quotient graph are given by $\mathcal{V}_{\mathcal{Q}} = \mathcal{V}_S \cup \bigcup_{i=k+1}^q o_i$. The set of edges in the quotient graph is defined as $\mathcal{E}_{\mathcal{Q}} = \{(u, v) \in \mathcal{E} \mid u, v \in \mathcal{V}_{\mathcal{Q}}\}$, containing all edges whose vertices are both in $\mathcal{V}_{\mathcal{Q}}$. The quotient graph is then given as $\mathcal{G}_{\mathcal{Q}} = (\mathcal{V}_{\mathcal{Q}}, \mathcal{E}_{\mathcal{Q}}, l|_{\mathcal{E}_{\mathcal{Q}}})$. The quotient graph is not unique, since any representatives of the sets $\{\text{orb}_{k+1}, \dots, \text{orb}_q\}$

can be chosen if the representatives ensure a connected quotient graph. For an explanatory example of the different quotient graphs, see the Supplementary Material (page 2).

3.4 Algorithm for finding the vertex orbits

The algorithm used to find the vertex orbits is called **Nauty** (short for **No AUTomorphism, Yes?**). Nauty is a test algorithm for graph isomorphisms designed by McKay (1981), see also McKay and Piperno (2014). Nauty works on both simple undirected graphs and directed graphs with loops, and returns a set of generators for the automorphism group. Since the composition of automorphisms is an automorphism itself, these generators can be used to find the automorphism group of a graph. When the automorphism group is known, the vertex orbits can be easily calculated. We used a simplified version of **Nauty** that is provided in the supplementary material. The simplified algorithm returns the vertex orbits of a graph instead of a set of generators. **Nauty** is explained along an example in the Supplementary Material (page 2).

4 Reducing networks

Theorem 1 derives a formula for determining the A-optimal weights in a network for random and non-random states. To compute the A-optimal weights, the influence matrix $\mathbb{X}$ has to be inverted, which can be computationally expensive for large networks. According to Sharma et al (2013), the Gauss-Jordan algorithm has a runtime of $O(I^3)$ to compute the inverse of a $I \times I$ matrix and this can be reduced to $O(I^{2.3727})$. However, the runtime for computing the matrix inverse cannot be lower than $O(I^2)$ since any algorithm must read I^2 entries, see e.g. Coppersmith and Winograd (1990). In addition, inverting matrices can be numerically unstable. The numerical instability increases with increasing dimensions of the inverted matrices.

Thus, inverting $\sqrt{I}$ many matrices with $(\sqrt{I} \times \sqrt{I})$ -dimension is computationally less expensive than inverting one $(I \times I)$ -dimensional matrix, for $\sqrt{I} \in \mathbb{N}$. Consequently, one idea is to partition the network into smaller ones, for example, into roughly $\sqrt{I}$ networks of size $\sqrt{I} \times \sqrt{I}$.

There exist different approaches to partitioning a graph into two (respectively k) smaller subgraphs. Among others, two popular algorithms are the maximum-cut and minimum-cut algorithms (see Commander (2008) and Karger (1993)). Whereas the maximum cut algorithm is obviously not suitable for the situation under consideration, as it seeks to maximize the total weight of removed edges, the minimum cut algorithm might be appropriate at first glance. This algorithm partitions a graph into subgraphs based on the minimum total weight of edges that must be removed. However, in our case, the weights are directly dependent on the influence matrix $\mathbb{X}$. Thus, there is no guarantee that the subgraphs produced by the minimum k -cut algorithm

are roughly of the same size. If this does not hold, the benefit of using the minimum k -cut algorithm would be limited, as the reduction in the largest network would be small.

Therefore, we use two approaches in which certain properties of the original network, such as symmetries, are used to reduce its size. The first approach is based on an algorithm originally proposed in Koutra (2017). This algorithm is based on the concept of quotient graphs and presented in detail in Section 4.1. While this approach only uses parts of the original network, namely the quotient graph, we propose a new partitioning approach that cuts the original network into subnetworks in a specific way. This procedure is explained in detail in Section 4.2.

4.1 Quotient graph

One method of reducing a network is to take advantage of the symmetry in the corresponding graph. Symmetries in a network are computed using the simplified *Nauty* algorithm. The reduced network has a reduced influence matrix and, therefore, the optimal design is computed faster and is computationally more stable. The first idea might be to use the skeleton of a graph, but this leads to two problems. First, there is no information about the symmetrical nodes, which are removed by reducing the graph to its skeleton. Since all vertices in the non-trivial vertex orbits are removed, the design of the skeleton does not include any of these vertices, making it difficult to assign a weight to those vertices. Second, a skeleton is not necessarily a connected graph, and therefore, the corresponding influence matrix might be singular. Hence, Theorem 1 would not be applicable. However, if one still wants to use the skeleton while incorporating some symmetrical nodes, the quotient graph provides a suitable alternative. Moreover, the quotient graph is connected, thereby resolving both issues associated with the skeleton. This approach builds up on the existing work regarding quotient graphs of Koutra (2017). To determine the weights assigned to the removed vertices during the quotient graph procedure, we can use Corollary 2, which is obtained from the following Theorem.

Theorem 2 *Consider a weighted graph $\mathcal{G}$, as well as an automorphism η on the graph $\mathcal{G}$. Let $\mathcal{G}'$ be the graph $\mathcal{G}$ under the automorphism η . Then the A -optimal design of the two networks for non-random states are identical, thus $\delta_{\mathcal{G}}^* = \delta_{\mathcal{G}'}^*$.*

Proof Let the influence matrix be $\mathbb{X}_{\mathcal{G}} = \mathbf{I}_{I \times I} + \mathcal{A}_w$ with $\mathcal{A}_w$ the weighted adjacency matrix of the graph $\mathcal{G}$. Furthermore, let $\mathcal{A}'_w$ be the weighted adjacency of the graph $\mathcal{G}'$ with the influence matrix $\mathbb{X}_{\mathcal{G}'} = \mathbf{I}_{I \times I} + \mathcal{A}'_w$. It follows from the definition of an automorphism η that the weighted adjacency matrix remains identical under the permutation so that $\mathcal{A}_w = \mathcal{A}'_w$ and thus $\mathbb{X}_{\mathcal{G}} = \mathbf{I}_{I \times I} + \mathcal{A}_w = \mathbf{I}_{I \times I} + \mathcal{A}'_w = \mathbb{X}_{\mathcal{G}'}$. Hence, the assertion follows from Theorem 1. $\square$

Theorem 2 states that if there exists an automorphism on a graph, the A-optimal design of the graph under the automorphism does not differ. If two vertices are in the same vertex orbit, there exists an automorphism that permutes these two vertices. This leads to the following result:

Corollary 2 *Let $\mathcal{G} = (\mathcal{V}, \mathcal{E}, l)$ be a weighted graph. Each pair of vertices $u, v \in \mathcal{V}$, which are contained in the same vertex orbit, is assigned the same weight in the A-optimal design of the graph $\mathcal{G}$.*

Consequently, the structure of the A-optimal design does not depend on the choice of vertices in the quotient graph. This fact leads to the following procedure for calculating a design based on the quotient graph.

Design algorithm 1 *A design based on the quotient graph follows this procedure:*

1. Calculate the vertex orbits of the graph $\mathcal{G}$.
2. Using the obtained vertex orbits, determine a quotient graph of the supergraph $\mathcal{G}$.
3. Derive the optimal design of the quotient graph denoted by δ_{quot}^* .
4. Add the removed vertices back into the design and assign them the same weight as the vertices in their respective orbits. This results in a vector δ_{quot} .
5. Rescale δ_{quot} with the factor $\frac{1}{\sum_{i=1}^I (\delta_{quot})_i}$, to obtain a suboptimal design δ_{quot} .

4.2 Partitioning the network into smaller subnetworks

Another approach to reduce the network size is to partition the network into smaller subnetworks. The idea is to split the graph into subgraphs that contain the vertices from specific vertex orbits. Thus, the subgraphs keep the symmetry of the supergraph with respect to the vertex orbits. The skeleton vertices are distributed so that they form subgraphs whose sizes are as equal as possible. Additionally, the algorithm ensures that the resulting subgraphs are connected. To formalize this complex procedure, an algorithm was developed that divides the graph into subgraphs. The self-developed algorithm is implemented in *R* and can be seen in the supplementary material. The function has 3 hyperparameters namely *range*, *max* and *iter.conv* their role in the algorithm will be explained later on. The algorithm requires two inputs: the skeleton and the influence matrix of the graph. The skeleton is derived using the simplified *Nauty* algorithm. The algorithm is illustrated by the following example, which involves an unweighted graph with 29 nodes. We don't present a pseudocode for the algorithm, since we just want to explore the reduction of networks. A network can be cut into subgraphs in multiple different ways. So we are rather interested in the idea of cutting a network instead of the performance of the cutting algorithm itself.

In the following Figures 1–10, the nodes are marked in different colors. The base colors are orange and blue. The blue vertices are the vertices which form the skeleton of the network, while the orange vertices correspond to the non-skeleton vertices. Furthermore, the figures contain green and purple vertices. Green vertices satisfy the criteria in the specific step of the algorithm, and purple vertices are their corresponding neighbors, which are merged with the green vertices to form a subgraph. The subgraphs in the current step of the algorithm are marked with black circles. The focus of this work is on the overall performance of the proposed procedure — that is, the relationship between network size reduction and the quality of the resulting designs — rather than on optimizing the cutting approach itself.

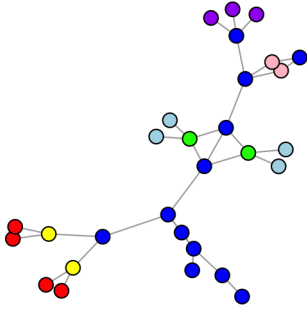


Fig. 1: Example graph

Fig. 1 shows the graph which is partitioned into subgraphs using the developed algorithm. The graph has 29 nodes. The different colors black, yellow, green, purple, light blue and pink indicate the different vertex orbits. The skeleton contains 12 vertices, which are colored in blue. Thus, the remaining 17 vertices are in non-trivial vertex orbits and have different symmetries with respect to the weights in the A-optimal design according to Theorem 2.

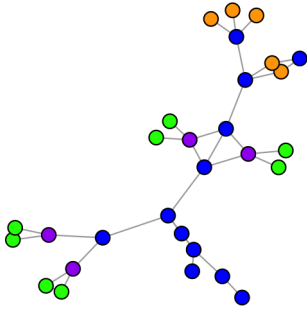


Fig. 2: Step 1

Step 1, see Fig. 2: The algorithm checks if there are any non-skeleton vertices connected only to other non-skeleton vertices (highlighted in green). Then, the algorithm merges those vertices with all their adjacent vertices, which are marked in purple. Consequently, all large structures of vertices that are not in the skeleton are merged into a subgraph.

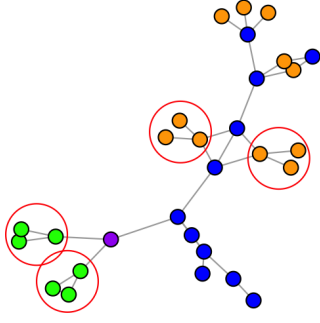


Fig. 3: Step 2

Step 2, see Fig. 3: The algorithm checks if any of the subgraphs are connected to a single skeleton vertex. The vertices of the subgraph that are connected to one skeleton vertex are colored in green, and the corresponding skeleton vertex is marked in purple. These vertices are fused into a subgraph.

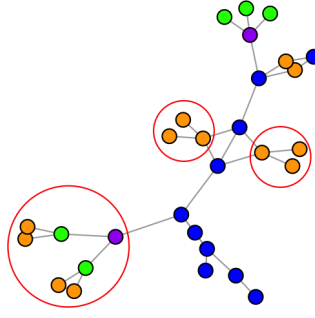


Fig. 4: Step 3

Step 3, see Fig. 4: The algorithm checks if any non-skeleton vertices are connected to a single skeleton vertex and combines them into a subgraph. The non-skeleton vertices are marked in green, while the adjacent skeleton vertex is marked in purple.

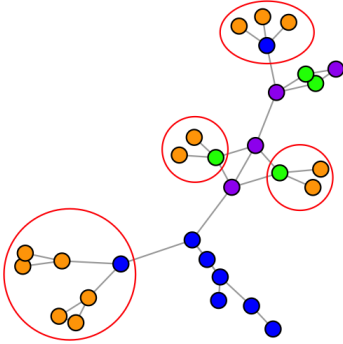


Fig. 5: Step 4

Step 4, see Fig. 5: The algorithm checks if any non-skeleton vertices are connected to multiple skeleton vertices. The non-skeleton vertices are marked in green, and the skeleton vertices are marked in purple. Now, the algorithm checks if any of the purple skeleton vertices are already in a subgraph.

If any skeleton vertex is in a subgraph, the depth of the subgraph, given as the number of vertices in the corresponding subgraph of the skeleton vertex, is calculated. If a purple skeleton vertex is not in a subgraph yet, the depth of the skeleton vertex is given by one. The skeleton with the minimal subgraph depth is chosen as the vertex, which is fused with its corresponding green non-skeleton vertex.

If none of the purple skeleton vertices are in a subgraph, the algorithm calculates the degree of the purple skeleton vertices with the weighted adjacency matrix and fuses the corresponding green non-skeleton vertex with the skeleton vertex of minimal degree. If the subgraph depth or degree is equal among multiple vertices, the algorithm chooses the vertex with the lower index.

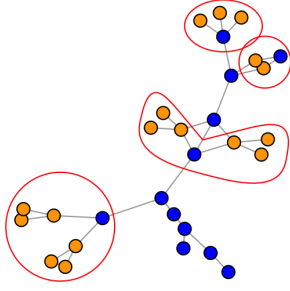


Fig. 6: Step 5

Step 5: In Fig. 6, the current state of the subgraphs in the algorithm after the initialization is shown. All non-skeleton vertices are merged into subgraphs. Now the while loop of the algorithm starts. The operations in the while loop are shown in the following Steps 5.1 – 5.3. The while loop ends after either all vertices are merged into subgraphs or a fixed number of iterations of the loop are run through. In the while loop, the skeleton vertices are distributed to form subgraphs of sizes which are as equal as possible.

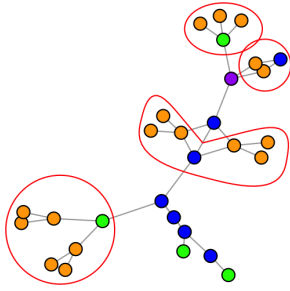


Fig. 7: Step 5.1

Step 5.1, see Fig. 7: In Step 5.1, the algorithm calculates the outer skeleton vertices. The green outer skeleton vertices are skeleton vertices that are connected to one skeleton vertex. The algorithm checks if the green outer skeletons are already part of a subgraph. In that case, the algorithm verifies if the subgraph including a green vertex is complete. If the subgraph is not complete, the algorithm merges the adjacent purple skeleton vertex into the subgraph. Otherwise, the algorithm does nothing.

A complete subgraph is a subgraph, which contains more than $\text{max} + \text{range}$ vertices. max and range are two parameters that can be specified before running the algorithm. If not specified, the default values of the parameters are $\text{max} = 5$ and $\text{range} = 1$.

If the green outer skeleton are not in a subgraph yet, the step 5.2 is applied.

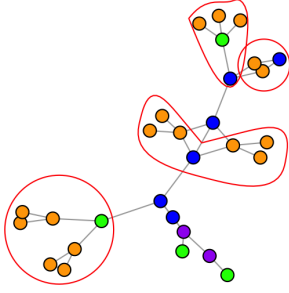


Fig. 8: Step 5.2

Step 5.2, see Fig. 8: In Step 5.2, the green outer skeleton vertices are not in a subgraph yet. A new subgraph is created containing only the green outer skeleton and their purple colored neighbors.

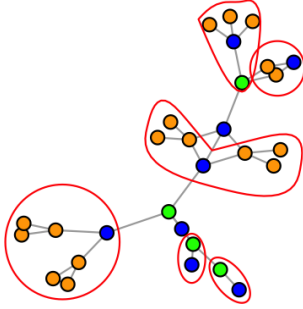


Fig. 9: Step 5.3

Step 5.3, see Fig. 9: The new outer skeleton vertices are given as the neighbors of the outer skeleton vertices. This means that the green new outer skeletons are the purple neighbors from Step 5.2. The loop returns to Step 5.1 using the given outer skeleton vertices if the stopping criteria are not met. The stopping criteria are fulfilled if either all vertices are in a subgraph or if the while loop 5.1 – 5.3 has run through `iter.conv` times.

The parameter `iter.conv` can be specified before running the algorithm, but has a default value of 20. If the while loop has reached `iter.conv` iterations, a new subgraph is created for each vertex that is not part of a subgraph yet. These newly created subgraphs contain only the corresponding vertices.

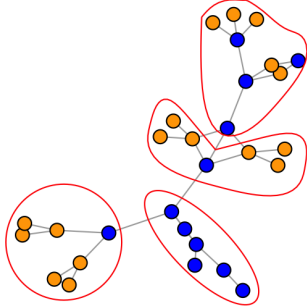


Fig. 10: Step 6

Step 6, see Fig. 10: The algorithm checks if all subgraphs have at least the size of `max - range`. If this criterion is not met, the subgraphs will be merged with the adjacent subgraph with the minimum subgraph depth. The minimum depth is calculated as in Step 4. The algorithm checks if all subgraphs are connected and returns the adjacency matrices of the subgraphs.

The application of the deterministic cutting algorithm leads to the following design procedure:

Design algorithm 2 *To obtain a design based on the deterministic cutting algorithm, follow this procedure:*

1. Use the cutting algorithm to obtain K subgraphs (K is a random value and not fixed).
2. Calculate the A -optimal design for each subgraph, denoted by $\delta_{sub,k}^*$ for $k \in \{1, \dots, K\}$.
3. Rescale each design $\delta_{sub,k}^*$ by the factor $\frac{I_k}{I}$, where the dimensions of the designs are $\dim(\delta_{sub,k}^*) = I_k$.
4. Fuse the rescaled vectors into a suboptimal design of size I , which is denoted by δ_{cut} .

5 Simulation study

5.1 Setting of the simulation study

In the setting of the paper, the optimal designs depend significantly on the network structure. As stated before, the structure of the considered network is represented by its adjacency matrix $\mathcal{A}$ and by the influence matrix $\mathbb{X}$. Thus, choosing a particular network structure corresponds to the choice of the corresponding influence matrix $\mathbb{X}$. For the simulation study, different choices of influence matrices are possible under the constraint of non-singularity. In order to illustrate the performance of the different reduction algorithms presented in Section 4, we use an influence matrix of the following form

$$\mathbb{X} = \mathbf{I}_{I \times I} + k\mathcal{D}^{-1/2}\mathcal{A}\mathcal{D}^{-1/2}, \quad (14)$$

where $\mathbf{I}_{I \times I}$ denotes the I -dimensional identity matrix, $\mathcal{A}$ the adjacency matrix of the graph, $\mathcal{D}$ the corresponding degree matrix and $k \in \mathbb{R}$.

The resulting influence matrix $\mathbb{X}$ contains ones on the diagonal entries and smaller values on the non-diagonals. The parameter k in (14) determines the extent of influence that adjacent nodes have on a node's state. For $k \in [0, 1)$, $\mathbb{X}$ is positive definite and therefore non-singular (see the proof in the supplementary material). Consequently, we choose a value of $k = 0.9$ so that the inverse of $\mathbb{X}$ exists, and adjacent nodes have a strong influence.

Table 1: The number of simulated networks, #Sim, by network size.

Network size I	$\{20, 30, \dots, 100\}$	200	300	400
#Sim	1000	500	300	200

For each network size I , we simulated #Sim networks where I and #Sim are depicted in Table 1. In each simulation step, a network of size I was generated using the R-code in the supplementary material. For the simulated

network, the analytical A -optimal design δ^* was calculated as the benchmark design using Corollary 1. Moreover, the design δ_{quot} based on the quotient graph algorithm and the design δ_{cut} based on the cutting algorithm were determined. Additionally, we considered the design δ_{deg} , in which the weight at a node is proportional to its degree, and the uniform design δ_{uni} , in which all weights are equal to $1/I$. We then calculated the A -efficiency of the designs $\delta = \delta_{\text{quot}}, \delta_{\text{cut}}, \delta_{\text{deg}}, \delta_{\text{uni}}$ given by

$$\text{Effi}_A(\delta) = \text{tr} \left([\mathbb{X}^\top \mathbf{D}_{\delta^*} \mathbb{X}]^{-1} \right) / \text{tr} \left([\mathbb{X}^\top \mathbf{D}_{\delta} \mathbb{X}]^{-1} \right).$$

This enabled us to compare the performance of the designs under consideration for different network structures and network sizes. The results of the simulation study are presented in the following section.

5.2 Results of the Simulation study

We begin by examining the A -efficiencies of the 10,000 simulated networks. Figure 11 provides the box plots of the resulting A -efficiencies across the simulated networks if the four different designs are used.

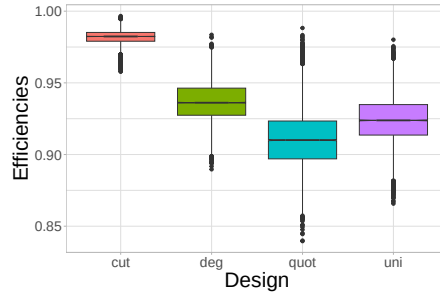


Fig. 11: A -efficiencies of the different designs

The box plot of the A -efficiencies of the design based on the cutting algorithm shows the highest efficiencies with the lowest variation, in particular the median is given by 0.982 and the minimum efficiency by 0.958. Thus, we can conclude that the design based on the cutting algorithm performs best with this influence matrix across many networks. We consider efficiencies above 80% as good efficiencies, and all the efficiencies lie above this threshold. Note that all A -efficiencies are high, which is due to the choice of the influence matrix. In another simulation with a different influence matrix, the efficiencies were significantly lower. In this setting, the design based on the cutting algorithm yielded the highest A -efficiencies. The Simulation study can be seen in the supplementary material. In the Figure 11, the quotient graph method results in designs with the lowest efficiencies.

To further study quotient-graph-based designs, we examined the relative number of remaining vertices. Figure 12 contains the relative amount of remaining vertices, based on the network size I . The box plots in Figure 13 illustrates that the amount of remaining vertices decreases with increasing network size. Furthermore, the median of remaining vertices is less than 40% for networks with size larger than 300, meaning that more than 60% of vertices are removed. The increase in removals is due to how the networks were simulated: as I increases, the network's symmetry increases. Figure 13 provides the A-efficiencies of the quotient graph design. The A-efficiencies behave similarly to the relative amount of not removed vertices. This behavior is expected, since the removal of a vertex is equivalent to the removal of information.

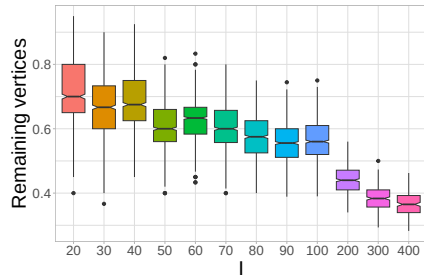


Fig. 12: Relative amount of not removed vertices in the quotient graph

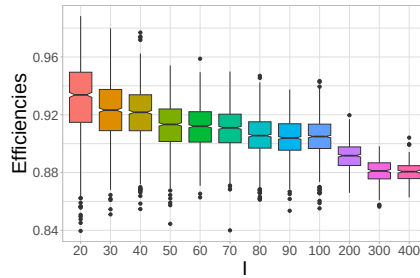


Fig. 13: A-efficiencies of the quotient graph.

6 Summary and discussion

In this work, we studied optimal designs for complex time-dependent networks. We introduced a linear random-effects model for time-dependent networks and derived the corresponding A-optimality criterion. Under a seasonality assumption, we proved that the problem reduces to one with constant states over time. In a second step, we considered the situation of large networks, as they occur in power distribution grids. In this case, computing the A-optimal design can be computationally difficult. Thus, we introduced two algorithms, namely the quotient graph and the cutting algorithm, to reduce the networks to smaller ones, for which the A-optimal designs can be calculated easily. In a final step, the smaller A-optimal designs were then generalized to the original networks, respectively. The performances of the different procedures were compared in a simulation study. We found that the designs based on the cutting algorithm yielded the highest A-efficiencies compared to the other procedures. Hence, dividing a network into multiple smaller networks and optimizing each locally proves to be a promising approach for large networks. However, the proposed partitioning algorithm could be further improved in terms of computational

robustness, reduced computation time, or improved design efficiency. In particular, the proposed cutting algorithm is currently mainly based on symmetries. One topic for further research is generalizing the cutting algorithm so that the cutting is done without incorporating symmetries.

We further studied the performance of the quotient graph algorithm in detail because its design performed poorly. We found that its poor performance was caused by highly symmetric networks, which led to many removed vertices. These highly symmetric networks were caused by the simulation method that was used for generating the networks. Therefore, additional simulation studies are needed to further analyze the designs using the quotient graph method. Moreover, future research may consider reducing its computation time, as well.

There are other algorithms that can be used for partitioning high dimensional networks, such as the minimum k -cut algorithm. The comparison of the developed cutting algorithm and these other partitioning methods was not within the scope of the paper and is left for future research.

A crucial assumption of the considered model is the separability of time and space, and the stochastically (in time) independence of the observations at the different nodes of the network. In a further step, we will reduce these assumptions and extend the model to one involving autoregressive correlation structures and non-separability.

Finally, the optimal designs are approximate designs that cannot be used directly for the optimal allocation of sensors. Thus, we will investigate how the obtained optimal designs can be discretized and how the design problem can be reformulated in a discrete optimization setup.

Supplement The Supplement contains how base functions can be constructed via Fourier series, an example for the simplified **Nauty** algorithm, a proof of the non-singularity of the influence matrix from the simulation study, and pseudo codes for the generation of the used networks and the simplified **Nauty** algorithm.

References

- Alexanderian A (2021) Optimal experimental design for infinite-dimensional Bayesian inverse problems governed by PDEs: a review. *Inverse Problems* 37(4):043001, DOI 10.1088/1361-6420/abe10c
- Bui T, Steiner SH, Stevens NT (2025) Genetic algorithm-based Bayesian optimal design for network experiments. *Technometrics* 0(0):1–20, DOI 10.1080/00401706.2025.2584500
- Cao M, Guo J, Xiao H, Wu L (2022) Reliability analysis and optimal generator allocation and protection strategy of a non-repairable power grid system. *Reliability Engineering & System Safety* 222:108443, DOI 10.1016/j.ress.2022.108443
- Commander C (2008) Maximum cut problem, max-cut DOI 10.1007/978-0-387-74759-0_358

- Cook RD, Nachtsheim CJ (1980) A comparison of algorithms for constructing exact d-optimal designs. *Technometrics* 22(3):315–324, DOI 10.1080/00401706.1980.10486162
- Coppersmith D, Winograd S (1990) Matrix multiplication via arithmetic progressions. *Journal of Symbolic Computation* 9(3):251–280, DOI [https://doi.org/10.1016/S0747-7171\(08\)80013-2](https://doi.org/10.1016/S0747-7171(08)80013-2)
- Du X, Engelmann A, Jiang Y, Faulwasser T, Houska B (2020) Optimal experiment design for AC power systems admittance estimation. *IFAC-PapersOnLine* 53(2):13311–13316, DOI 10.1016/j.ifacol.2020.12.163, 21st IFAC World Congress
- Duan P, He L, Huang L, Chen G, Shi L (2022) Sensor scheduling design for complex networks under a distributed state estimation framework. *Automatica* 146:110628, DOI 10.1016/j.automatica.2022.110628
- Fedorov VV (2013) *Theory of optimal experiments*. Elsevier
- Harary F (1969) *Graph Theory*. Addison-Wesley, Reading, MA
- Jiang Y, Gómez AME (2025) An adaptive sampling strategy for online monitoring of partially observed networks. *Technometrics* 0(ja):1–20, DOI 10.1080/00401706.2025.2580634
- Karger DR (1993) Global min-cuts in rnc, and other ramifications of a simple min-cut algorithm. In: *Proceedings of the Fourth Annual ACM-SIAM Symposium on Discrete Algorithms, Society for Industrial and Applied Mathematics, USA, SODA '93*, p 21–30
- Koutra V (2017) *Designing experiments on networks*. PhD thesis, University of Southampton
- Koutra V, Gilmour SG, Parker BM (2021) Optimal block designs for experiments on networks. *Journal of the Royal Statistical Society Series C: Applied Statistics* 70(3):596–618, DOI 10.1111/rssc.12473
- Li Q, Negi R, Ilić MD (2011) Phasor measurement units placement for power system state estimation: A greedy approach. In: *2011 IEEE Power and Energy Society General Meeting*, pp 1–8, DOI 10.1109/PES.2011.6039076
- McKay BD (1981) Practical graph isomorphism. *Congressus Numerantium* 30:45–87
- McKay BD, Piperno A (2014) Practical graph isomorphism, ii. *Journal of Symbolic Computation* 60:94–112, DOI <https://doi.org/10.1016/j.jsc.2013.09.003>, URL <https://www.sciencedirect.com/science/article/pii/S0747717113001193>
- Müller CH (2026) Optimal designs for the Kalman filter applied to electrical power distribution grid. Submitted
- Müller CH, Schorning K (2023) A-optimal designs for state estimation in networks. *Statistical Papers* 64(4):1159–1186
- Rahman S (2017) *Basic graph theory*. Springer
- Schlösser T, Angioni A, Ponci F, Monti A (2014) Impact of pseudo-measurements from new load profiles on state estimation in distribution grids. In: *2014 IEEE International Instrumentation and Measurement Technology Conference (I2MTC) Proceedings*, pp 625–630, DOI 10.1109/I2MTC.2014.6860819

- Sharma G, Agarwala A, Bhattacharya B (2013) A fast parallel Gauss Jordan algorithm for matrix inversion using CUDA. *Computers & Structures* 128:31–37
- Singhal H, Michailidis G (2008) Optimal sampling in state space models with applications to network monitoring. In: *Proceedings of the 2008 ACM SIGMETRICS International Conference on Measurement and Modeling of Computer Systems*, Association for Computing Machinery, New York, NY, USA, SIGMETRICS '08, p 145–156, DOI 10.1145/1375457.1375474
- Uciński D (2022) E-optimum sensor selection for estimation of subsets of parameters. *Measurement* 187:110286, DOI 10.1016/j.measurement.2021.110286
- Uciński D (2025) A-optimal sensor location subject to correlated observations via a fast exchange algorithm. *Measurement* 254:117794, DOI <https://doi.org/10.1016/j.measurement.2025.117794>, URL <https://www.sciencedirect.com/science/article/pii/S0263224125011534>
- Uciński D, Patan M (2024) Sensor selection with correlated observations via convex relaxation. In: *2024 IEEE 63rd Conference on Decision and Control (CDC)*, pp 2703–2708, DOI 10.1109/CDC56724.2024.10886743
- Umeyama S (1988) An eigendecomposition approach to weighted graph matching problems. *IEEE Transactions on Pattern Analysis and Machine Intelligence* 10(5):695–703
- Xygkis TC, Korres GN, Manousakis NM (2018) Fisher information-based meter placement in distribution grids via the D-optimal experimental design. *IEEE Transactions on Smart Grid* 9(2):1452–1461, DOI 10.1109/TSG.2016.2592102
- Zhang Q (2025) Rerandomization algorithms for optimal designs of network A/B tests. *Technometrics* 67(4):655–668, DOI 10.1080/00401706.2025.2505438